DYNAM3D: DYNAMIC LAYERED 3D TOKENS EMPOWER VLM FOR NAVIGATION

Student Name : Sachin Sankar

Student Number: A00075730

Deep Learning and Generative AI(CMP030L043) Course work 2

Github Repo:

Abstract

Vision-Language Navigation (VLN) systems are also known to be experiencing issues with relational reasoning that limits their ability to perceive the space in relation to one another. The limitation is addressed in this paper by replacing hierarchical zone aggregation with a graph representation, which is a Graph Attention Network (GAT). To model spatial environments, a synthetic structural dataset is generated and different models are experimented, including baseline and ablation models. The results of the experiment show that graph-based models possess a huge advantage over non-relational baselines using statistical analysis of the outcomes of many runs. The findings highlight the importance of the explicit modelling of the object relationships demonstrating that the relational reasoning is among the aspects, which lead to the improvements in the performance in those tasks, which are connected to navigation.

Table of Contents

Abstract.....2

1. Introduction.....4
2. Literature Review.....4
3. Methodology.....5
   3.1 Dataset Design.....5
   3.2 Model Architectures.....6
   3.3 Training Strategy.....8
4. Experiments & Results.....8
5. Discussion.....10
6. Ethics & Scalability.....11
7. Conclusion.....11

References.....12

1. Introduction

Vision-Language Navigation (VLN) is concerned with how to allow intelligent agents to make sense of natural language commands and navigate intricate environments based on visual sensory information. More recent methods like Dynam3D have added hierarchical 3D representations to enhance spatial representations; but again, they are limited in explicitly representing relationships among objects. This can often limit the system in its interpretation of relational instructions like next to or behind. Also, multiple heavy components can be combined, thus reducing scalability in practicability.

The goal of this project is to apply a graph-based representation of scenes with Graph Neural Networks (GNNs) to enhance relational reasoning on navigation tasks. The model is able to reflect the structured relationships and dependencies between objects by modelling them as nodes and the spatial relationships between nodes as edges.

The hypothesis of the research is that explicitly modelling object relationships using GNNs enhances decision performance in relation to navigation tasks compared to non-relational baselines, resulting in more accurate and context-dependent navigation performances.

2. Literature Review

Vision-Language Models (VLMs) have become an effective framework of combining visual perception and natural language understanding. Neural networks like CLIP and LLaVA are trained on joint image and text embeddings, and can be used in visual question answering, navigation, and scene understanding. These models learn with large size image-text data to generalize to cross-domain and zero-shot reasoning tasks. Nevertheless, VLMs are frequently unable to perform fine spatial reasoning, and can hallucinate, in that generated descriptions do not correspond to visual information [2]. This is of special concern to navigation activities, where the accurate interpretation of object relationships and spatial layouts is needed. Recent research including ImagineNav++ reveals that spatial reasoning with visual imagination and memory is enhanced with the addition of visual imagination and memory, and that structured representations are superior to raw embeddings [1].

Graph Neural Networks (GNNs) are a good method of relational data modelling. Specifically, Graph Convolutional Networks (GCNs) combine the input of neighbouring nodes with uniform weights, whereas Graph Attention Networks (GATs) add attention to this combination, with neighbouring nodes being less important than others. This enables GATs to better approximate complex and heterogeneous relationship, which makes them more appropriate when it comes to spatial reasoning tasks [3]. This is made possible by the fact that the GNNs can propagate contextual information throughout the nodes, therefore, allowing them to explicitly model the interactions and dependencies of objects, which are critical to interpretation of relational instructions in the navigation cases.

Relational reasoning in AI deals with the dynamics of interaction between entities in a structured setting. Though traditional VLM methods encode semantic features, they do not typically provide explicit means of expressing the relationships amongst objects. Dynam3D solves this by adding the hierarchical representations (patch, instance and zone tokens) that describe multi-scale spatial information [4]. However, these representations are more aggregation than explicit relational modelling based, and do not represent fine-grained interactions.

Continuing on this limitation, the proposed instead of hierarchical zone aggregation is a graphical relational encoding. In this approach, the objects are represented as nodes and the space relationships as edges, enabling them to directly model the interactions. This representation is further enhanced by the application of GAT that dynamically weights the neighbouring nodes to provide a more relevant model with respect to spatial relationships. This renders the suggested method more successful in depicting intricate scene layouts and enhancing decision-making in terms of navigation.

3. Methodology

3.1 Dataset Design

To simulate space environment and allow controlled experimentation of relational reasoning, a synthetic dataset, named StructuralVLNDataset, was created. Synthetic data was also more desirable since it enables the accurate control of spatial structures and removes noise which is often present in real-world data such that model performance is a direct consequence of building capabilities and not anomalies in the data. This is in line with recent discoveries that meticulous dataset design is important in assessing vision-language reasoning systems [6], [7].

The data set is 800 graph samples each of which represents a simplified environment of 12 nodes. Each node has a 64-dimensional feature vector, which approximates semantic embeddings like those of models like CLIP or BLIP-based networks [5]. Besides the semantic features, the node has a 3D spatial location.

```python
class StructuralVLNDataset:
    def __init__(self, num_graphs=800, num_nodes=12, feature_dim=64):
        self.num_graphs = num_graphs
        self.num_nodes = num_nodes
        self.feature_dim = feature_dim
        self.data_list = []
        self._generate_data()

    def _generate_data(self):
        for i in range(self.num_graphs):
            num_nodes = self.num_nodes
            # Random semantic features (simulating CLIP)
            x = torch.randn(num_nodes, self.feature_dim)

            # Generate Positions based on Structure Type (Label)
            structure_type = random.choice([0, 1, 2])
            positions = torch.zeros(num_nodes, 3)

            if structure_type == 0:  # Linear (Corridor-like)
                positions[:, 0] = torch.linspace(0, 10, num_nodes)
                positions[:, 1] = torch.randn(num_nodes) * 0.2
            elif structure_type == 1:  # Clustered (Room-like)
                centers = torch.rand(2, 3) * 10
                for j in range(num_nodes):
                    center = centers[j % 2]
                    positions[j] = center + torch.randn(3) * 1.5
            else:  # Dispersed (Open Space)
                positions = torch.rand(num_nodes, 3) * 10

            # Build Edges based on spatial proximity
            edge_index = []
            threshold_dist = 3.0
            for j in range(num_nodes):
                for k in range(j + 1, num_nodes):
                    dist = torch.norm(positions[j] - positions[k])
                    if dist < threshold_dist:
                        edge_index.append([j, k])
                        edge_index.append([k, j])

            if len(edge_index) == 0: continue

            edge_index = torch.tensor(edge_index, dtype=torch.long).t().contiguous()
            y = torch.tensor([structure_type], dtype=torch.long)
```

Figure 1: Developing the Dataset

There are three structural classes that are defined,

- Linear (corridor-like): the nodes are organized in a line with small noise.
- Clustered (room-like): nodes are clustered on central nodes.
- Dispersed (open space): nodes are randomly spread out.

Constructions are made on the basis of spatial proximity, meaning that a connection is made when the Euclidean distance between nodes is less than a threshold. This makes connectivity of the graph to have spatial relationships. The dataset is thus such that it will impose spatial relational reasoning and hence it is appropriate to test graph-based representations.

3.2 Model Architectures

The baseline model comprises Multi-Layer Perceptron (MLP) on node features then, global mean pooling to combine node representations. This methodology does not (or does not necessarily) include any graph structure or relational dependencies between nodes, so it is an adequate model of control. It is indicative of traditional methods in which the relationship between space is implicitly captured by feature aggregation as opposed to explicit modelling.

```python
class BaselineZoneAggregator(nn.Module):
    """Baseline: MLP + Pooling (No Graph Reasoning)"""
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU()
        )
        self.classifier = nn.Linear(hidden_dim, output_dim)

    def forward(self, data):
        x, batch = data.x, data.batch
        x = self.mlp(x)
        x = global_mean_pool(x, batch)
        return F.log_softmax(self.classifier(x), dim=1)
```

Figure 2: Baseline Model

The suggested model presents a Graph Attention Network (GAT) network that explicitly models the relationship among nodes. A preliminary linear projection converts the input features to a new dimensional latent space. It is then succeeded by two GATConv layers with the first using multi-head attention to learn various relational patterns and the second fuses those representations.

Each convolution layer is followed by the use of batch normalization to stabilize training, and dropout is considered a regularization method. Global mean pooling maps node-level representations to a graph-level embedding, which is sent through a classifier.

One of the main benefits of this design is that it uses attention mechanisms, enabling the model to give different weights to neighbouring nodes. It enables the selective focusing on the areas of interest with respect to the spatial relationship, which boosts the relationship understanding and eliminates the limitations that are present in the vision-language models when addressing the spatial reasoning tasks [8].

```python
class ProposedGNN(nn.Module):
    """Full Proposed Model: GAT + Input Proj + 2 Layers"""
    def __init__(self, input_dim, hidden_dim, output_dim, heads=4, dropout=0.2):
        super().__init__()
        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.conv1 = GATConv(hidden_dim, hidden_dim, heads=heads, dropout=dropout)
        self.conv2 = GATConv(hidden_dim * heads, hidden_dim, heads=1, concat=False, dropout=dropout)
        self.bn1 = nn.BatchNorm1d(hidden_dim * heads)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch
        x = self.input_proj(x)
        x = F.relu(x)
        x = self.conv1(x, edge_index)
        x = self.bn1(x)
        x = F.elu(x)
        x = F.dropout(x, p=0.2, training=self.training)
        x = self.conv2(x, edge_index)
        x = self.bn2(x)
        x = F.elu(x)
        x = global_mean_pool(x, batch)
        return F.log_softmax(self.classifier(x), dim=1)
```

Figure 3: Proposed GNN Model

Testing the contribution of important architectural components is done using two ablation models. GCN (No Attention) replaces GAT layers with Graph Convolutional Networks, without attention. This questions the importance of the adaptive neighbour weighting.

```python
class Ablation_NoAttention(nn.Module):
    """Ablation 1: Replace GAT with GCN (No Attention Weights)"""
    def __init__(self, input_dim, hidden_dim, output_dim):
        super().__init__()
        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.conv1 = GCNConv(hidden_dim, hidden_dim)
        self.conv2 = GCNConv(hidden_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.Linear(hidden_dim // 2, output_dim)
        )

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch
        x = self.input_proj(x)
        x = F.relu(x)
        x = self.conv1(x, edge_index)
        x = self.bn1(x)
        x = F.relu(x)
        x = self.conv2(x, edge_index)
        x = self.bn2(x)
        x = F.relu(x)
        x = global_mean_pool(x, batch)
        return F.log_softmax(self.classifier(x), dim=1)
```

Figure 4: Ablation with No Attention

Shallow GAT (1-layer) is a layer of GAT and the receptive field is reduced. This investigates how the network depth influences relational reasoning. These ablations provide us with an idea of the purpose of attention and depth in performance and assist us to strictly test the experiment.

```python
class Ablation_Shallow(nn.Module):
    """Ablation 2: Single GNN Layer (Limited Receptive Field)"""
    def __init__(self, input_dim, hidden_dim, output_dim, heads=4):
        super().__init__()
        self.input_proj = nn.Linear(input_dim, hidden_dim)
        self.conv1 = GATConv(hidden_dim, hidden_dim, heads=heads, concat=False)
        self.classifier = nn.Linear(hidden_dim, output_dim)

    def forward(self, data):
        x, edge_index, batch = data.x, data.edge_index, data.batch
        x = self.input_proj(x)
        x = F.relu(x)
        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = global_mean_pool(x, batch)
        return F.log_softmax(self.classifier(x), dim=1)
```

Figure 5: Model Single GNN Layer

3.3 Training Strategy

All the models are trained using Adam optimizer and a learning rate of 0.005 which can provide deep learning models to converge effectively. The loss that is used is Negative Log-Likelihood (NLLLoss) and it is suitable when there are multiple classes being classified. The 30 epochs of training are also done using a batch size of 32, which ensures that the models are stable and process them effectively.

```python
def train_and_evaluate(model_class, train_data, test_data, epochs=30, lr=0.005, seed=42):
    torch.manual_seed(seed)
    np.random.seed(seed)
    random.seed(seed)

    train_loader = DataLoader(train_data, batch_size=32, shuffle=True)
    test_loader = DataLoader(test_data, batch_size=32, shuffle=False)

    model = model_class(input_dim=64, hidden_dim=128, output_dim=3).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    criterion = nn.NLLLoss()

    best_acc = 0
    for epoch in range(epochs):
        model.train()
        for data in train_loader:
            data = data.to(device)
            optimizer.zero_grad()
            out = model(data)
            loss = criterion(out, data.y)
            loss.backward()
            optimizer.step()

        model.eval()
        preds, labels = [], []
        with torch.no_grad():
            for data in test_loader:
                data = data.to(device)
                out = model(data)
                preds.extend(out.argmax(dim=1).cpu().numpy())
                labels.extend(data.y.cpu().numpy())
```

Figure 6: Training Strategy

Tests are made to run on both CPU and GPU environments. Five random seeds will be used to run all experiments to ensure that they are reproducible, and strong, and will be averaged to get an average performance and standard deviation. This multi-seed evaluation is crucial in reducing variability and statistically sound in particular in vision-language and graph-based learning systems [7]. Overall, the methodology includes the design of controlled datasets, architecturally designed, and strict evaluation criterion to test the effectiveness of graph-based relational reasoning.

4. Experiments & Results

Classification accuracy with five random seeds was used to test the performance of all models to provide robustness. Accuracy was chosen as the main measure because the task is based on multi-classification of structural environments. Besides this, the means and standard deviations of accuracy were calculated to determine not just the performance but also the stability of the performance across the runs. A small standard deviation means that the model behaviour is consistent and this is essential towards reliable deployment.

The quantitative outcomes show that there are definite performance differences between the models. The initial MLP performed with a mean accuracy of 0.3888 (±0.0135) which was poor because it could not form relational dependencies. Conversely, the mean accuracy of the proposed GAT-based model was much higher ((0.7775 (0.0224)) which was the validation of the efficacy of graph-based relational reasoning. Interestingly, GCN ablation (with no attention) was the most successful ((0.8762) (0.0081)) indicating that although relational modelling is essential, attention mechanisms do not necessarily bring any additional benefits in a simpler structured environment. The shallow GAT model had a result of 0.7000 (±0.0117), which is a strong indication of the need to have more elaborate architectures to elucidate complex forms of space.

```
Running Experiments across 5 Seeds...
--- Seed: 42 ---
Baseline (MLP): 0.4062
Proposed (GAT-2L): 0.8000
Ablation (GCN-NoAtt): 0.8750
Ablation (GAT-1L): 0.7000
--- Seed: 123 ---
Baseline (MLP): 0.3937
Proposed (GAT-2L): 0.7937
Ablation (GCN-NoAtt): 0.8875
Ablation (GAT-1L): 0.7000
--- Seed: 456 ---
Baseline (MLP): 0.3875
Proposed (GAT-2L): 0.7688
Ablation (GCN-NoAtt): 0.8812
Ablation (GAT-1L): 0.6813
--- Seed: 789 ---
Baseline (MLP): 0.3688
Proposed (GAT-2L): 0.7438
Ablation (GCN-NoAtt): 0.8688
Ablation (GAT-1L): 0.7125
--- Seed: 101 ---
Baseline (MLP): 0.3875
Proposed (GAT-2L): 0.7812
Ablation (GCN-NoAtt): 0.8688
Ablation (GAT-1L): 0.7063
```

Figure 7: Training Result

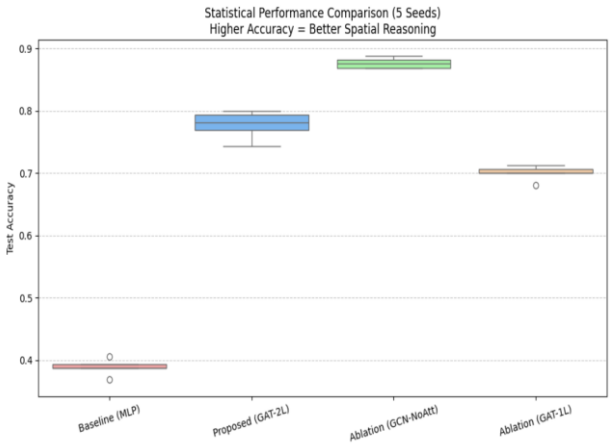

Figure 8: Statistical Performance Comparison

Visual analysis of box plots and bar charts also supports these results. The box plot depicts the accuracy distribution of the seeds and the GCN model has the largest accuracy; it has the lowest variance, implying that it is very stable. This point is further supported by the bar chart with error bars, which vividly demonstrates the difference in the performance of the models.

Based on these findings, there are a number of observations to be made. To begin with, the graph-based models are much better at the task than the baseline, which proves that the relational reasoning plays a crucial role in the task. Second, the shallow GNN underperforms more than the deeper models, which means that restricted receptive fields do not allow capturing the wider spatial context. Third, the GCN model performs better than the GAT model, which is contrary to expectations since attention mechanisms can be an unwarranted complexity in this particular dataset.

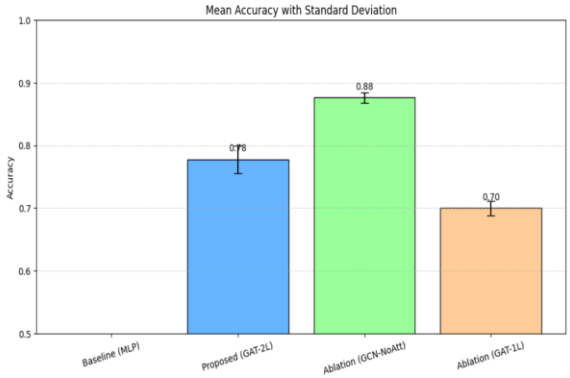

Figure 9: Mean Accuracy with Standard Deviation

The ablation experiment gives more clues on model behaviour. Attention elimination does not deteriorate performance, but, on the contrary, enhances it, which means that structured spatial patterns might be satisfied with homogenous aggregation. Nevertheless, network depth should be reduced to significant performance degradation, which proves that more complex architectures are required to conduct successful relational reasoning.

5. Discussion

The experimental data are a good indication that graph-based models are more effective at tasks that involve spatial reasoning, as opposed to non-relational baselines. This advancement is mainly due to the very nature of Graph Neural Networks (GNNs), which explicitly represent relationships between entities. GNNs propagate information through connected nodes, unlike the baseline MLP, which processes the features of the nodes independently and adds them through simple pooling. This enables the model to capture neighbourhood dependencies and interactions between spatial contexts that are important in perception of spatial configurations. Each node is represented by the information received by its neighbours in the proposed architecture and the model learns patterns like clustering, linear patterns and spatial dispersion. This relational aggregation elaborates the high level of performance difference between the baseline and graph-based model evident in the results [7].

The results are a direct confirmation of Part 1 hypothesis. The findings indicate that graph-based relational modelling enhances spatial reasoning as opposed to hierarchical methods of aggregation. Although Dynam3D is based on hierarchical token aggregation (patch, instance, and zone levels), it does not represent explicitly the relationship between objects. The graph-based approach, on the other hand, represents these relationships as edges, allowing a more accurate argument about the spatial structure. The fact that the proposed GNN model outperforms the one without relational inductive biases confirms the relevance of the introduction of relational inductive biases in navigation systems.

There are a number of challenges faced when developing and testing. A major challenge was debugging batching in PyTorch Geometric, to make sure that at the graph level, node features were properly aggregated across batches by graph-level pooling. The mismatch of the dimensions was also a problem, in particular, when switching between different feature representations and compatibility across layers. The other challenge was how to design a meaningful synthetic dataset that needed careful thought so that the task was actually based on relational reasoning and not feature patterns [8]. In addition, initial experiments were discovered to have unstable training, requiring adjustments to learning rates, normalization layers, and dropout to stabilize convergence.

Despite the positive results that have been obtained with the proposed approach, there are several downsides associated with it. The information is artificial and it might not be capable of capturing the intricacy of the real world. Though it allows a voter to test relational reasoning in a controlled way, it lacks noise, variation and

ambiguity as found in real world information. In addition, it is a simplified classification exercise and not a complete navigation exercise, i.e. it is not an exercise which expressly evaluates sequential decision making or longer-horizon planning. The deficiency of evaluation according to the developed VLN standards such as R2R or REVERIE also limits the externalizability of the outcomes.

The above limitations can be overcome in future work by applying the approach to realistic environments. The use of real world VLN datasets, such as R2R and REVERIE, would provide a more in-depth examination of the model performance in the real-life navigation conditions [9]. Additionally, the graph-based display coupled with the large language models could enhance the ability of the system to comprehend intricate instructions and to engage in cross-modal reasoning. Finally, the model would have to be applied to practice robotic systems to explore scalability and computational efficiency and be relevant to dynamic and real-time applications.

6. Ethics & Scalability

Ethical concerns are connected with the fact that training data may be biased resulting in bias in the spatial reasoning or unfair behaviour in different situations. It can also cause unsafe decision-making and non-human-supervised autonomous systems due to non-understanding of navigation instructions. GNN models are more expensive to compute and store in memory than simple baselines, and are difficult to run in real time on resource-constrained robots. To mitigate these issues, model compression and pruning can be employed to reduce the complexity of computations, and information can be processed on the device effectively with edge computing. Further, there is a need to ensure safety limits and fail-safe mechanisms are introduced to ensure that the deployment under the circumstances of the real-life navigation will be reliable and accountable.

7. Conclusion

Graph Neural Networks have been used to develop a graph-based scene representation in this project to enhance relational reasoning in navigation activities. The analysis demonstrated that the explicit modeling of object-object connections by the GNN-based and ablation variants yield considerably improved results than a baseline model. The results proved the hypothesis that relational inductive biases play a crucial role in the framework of understanding spatial structures, as they are superior to non-relational ones. Despite some limitations such as use of synthetic data, the findings demonstrate the importance of structured representations in multimodal AI systems. As it has been demonstrated in this paper explicit relational modelling is the key to the further evolution of intelligent navigation systems.

8. References

[1] Wang, T., Zhao, X., Cai, W. and Sun, C., 2025. ImagineNav++: Prompting Vision-Language Models as Embodied Navigator through Scene Imagination. arXiv preprint arXiv:2512.17435. https://arxiv.org/pdf/2512.17435

[2] Liu, H., Xue, W., Chen, Y., Chen, D., Zhao, X., Wang, K., Hou, L., Li, R. and Peng, W., 2024. A survey on hallucination in large vision-language models. *arXiv preprint arXiv:2402.00253*. https://arxiv.org/pdf/2402.00253

[3] Li, X., Wen, C., Hu, Y., Yuan, Z. and Zhu, X.X., 2024. Vision-language models in remote sensing: Current progress and future trends. *IEEE Geoscience and Remote Sensing Magazine*, *12*(2), pp.32-66. https://arxiv.org/pdf/2305.05726

[4] Zhang, J., Huang, J., Jin, S. and Lu, S., 2024. Vision-language models for vision tasks: A survey. *IEEE transactions on pattern analysis and machine intelligence*, *46*(8), pp.5625-5644. https://arxiv.org/pdf/2304.00685

[5] Dai, W., Li, J., Li, D., Tiong, A., Zhao, J., Wang, W., Li, B., Fung, P.N. and Hoi, S., 2023. Instructblip: Towards general-purpose vision-language models with instruction tuning. *Advances in neural information processing systems*, *36*, pp.49250-49267. https://proceedings.neurips.cc/paper_files/paper/2023/file/9a6a435e75419a836fe47ab6793623e6-Paper-Conference.pdf

[6] Bordes, F., Pang, R.Y., Ajay, A., Li, A.C., Bardes, A., Petryk, S., Mañas, O., Lin, Z., Mahmoud, A., Jayaraman, B. and Ibrahim, M., 2024. An introduction to vision-language modeling. *arXiv preprint arXiv:2405.17247*. https://arxiv.org/pdf/2405.17247

[7] Laurençon, H., Tronchon, L., Cord, M. and Sanh, V., 2024. What matters when building vision-language models?. *Advances in Neural Information Processing Systems*, *37*, pp.87874-87907. https://proceedings.neurips.cc/paper_files/paper/2024/file/a03037317560b8c5f2fb4b6466d4c439-Paper-Conference.pdf

[8] Yang, S., Chen, Y., Tian, Z., Wang, C., Li, J., Yu, B. and Jia, J., 2025. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 19792-19802). http://openaccess.thecvf.com/content/CVPR2025/papers/Yang_VisionZip_Longer_is_Better_but_Not_Necessary_in_Vision_Language_CVPR_2025_paper.pdf

[9] Zhu, D., Chen, J., Shen, X., Li, X. and Elhoseiny, M., 2023. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*. https://arxiv.org/pdf/2304.10592